UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA, ELETTRONICA E SISTEMISTICA

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA

ELETTRONICA DIGITALE

RELAZIONE SECONDO PROGETTINO – MINI ALU A N BIT

Petrassi Maria Jose' – 251439

Ponte Andrea – 251050

Ruffolo Emilio – 250521

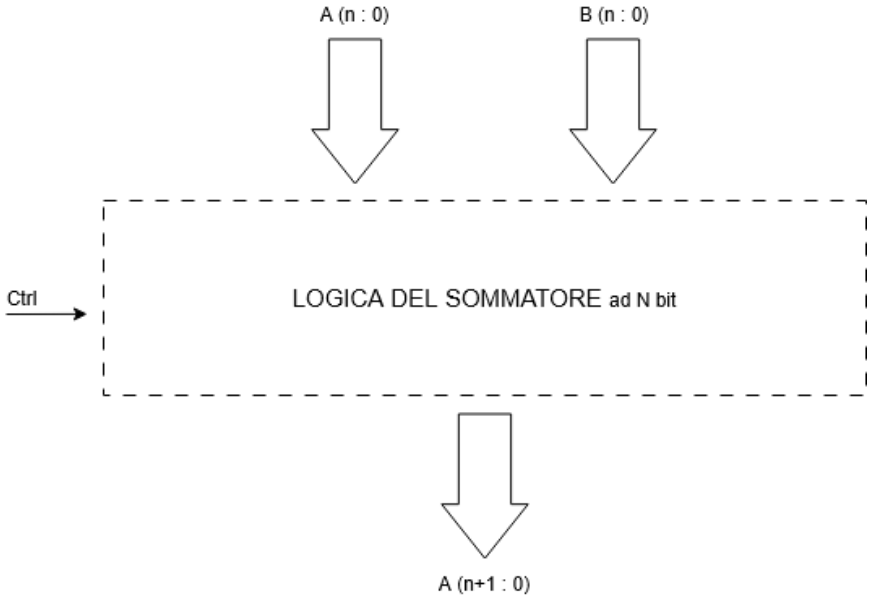
Traccia Del Progetto

Il presente progetto richiede la progettazione e l'implementazione in linguaggio VHDL di un'Unità Aritmetica Combinatoria destinata all'esecuzione selettiva di operazioni di somma o differenza su operandi numerici in formato complemento a due. Il circuito dovrà operare su dati di dimensione n bit. L'unità sarà dotata di due ingressi operandi (A e B) e di un ingresso di controllo a 1 bit (Ctrl). La funzionalità del circuito è definita come $RISULTATO = A + B$ quando $Ctrl = 0$ e $RISULTATO = A - B$ quando $Ctrl = 1$. È un requisito fondamentale che il progetto implementi la corretta gestione dell'aritmetica in complemento a due, inclusa la gestione della condizione di overflow.

La fase di caratterizzazione e validazione del progetto deve essere condotta eseguendo simulazioni e analisi su due diversi dimensionamenti del circuito, come di seguito specificato:

- Fissato il parametro $n = 8\ bit$, è richiesta l'esecuzione della simulazione esaustiva tramite la creazione di un TestBench che dimostri il corretto funzionamento del circuito verificando tutte le possibili combinazioni di ingresso, inclusi i casi limite di rappresentazione e l'occorrenza di overflow.
- Fissato invece il parametro $n = 16\ bit$, non è richiesta la simulazione esaustiva. È sufficiente un TestBench che dimostri la correttezza funzionale attraverso l'analisi di alcuni stati significativi come la verifica dell'overflow.

Per le implementazioni è richiesta l'analisi dettagliata delle risorse hardware utilizzate in fase di sintesi (es. numero di celle logiche, LUT, registri) e una valutazione documentata della dissipazione di potenza del circuito in condizioni operative del circuito ad $8\ bit$. La relazione tecnica finale dovrà contenere i risultati completi di entrambe le campagne di simulazione e delle analisi hardware.



1. Architettura del Circuito

Il cuore del progetto è l'unità aritmetica combinatoria parametrica **RCAluG** (*Ripple Carry ALU Generic*), progettata per eseguire operazioni di somma e differenza su numeri interi in complemento a due. L'architettura combina la semplicità strutturale della propagazione del riporto in cascata con l'uso formale delle funzioni *Generate* e *Propagate*.

1.1 Logica Aritmetica (Propagate & Generate)

Il circuito implementa le equazioni caratteristiche per la gestione dei riporti. Per ogni stadio i, vengono calcolati due segnali ausiliari:

- **Generate (G_i):** Indica se lo stadio genera riporto indipendentemente dall'ingresso ($A_i \cdot B_i$)
- **Propagate (P_i):** Indica se lo stadio propaga il riporto in ingresso ($A_i \oplus B_i$)

Il calcolo del vettore dei riporti C avviene secondo l'equazione ricorsiva:

$$C_{i+1} = G_i + (P_i \cdot C_i)$$

Questa struttura, sebbene descritta tramite vettori per compattezza del codice VHDL, si traduce a livello hardware in una catena di dipendenze tipica di un Ripple Carry Adder, dove il calcolo del bit i-esimo dipende dalla disponibilità del riporto dello stadio $i - 1$.

1.2 Gestione delle Operazioni e del Segno

Il circuito è controllato da un segnale a 1 bit (Ctrl) che definisce l'operazione:

- **Addizione ($Ctrl = 0$):** $A + B$
- **Sottrazione ($Ctrl = 1$):** $A - B$

La sottrazione è realizzata sfruttando la proprietà del complemento a due ($A - B = A + \bar{B} + 1$), ottenuta invertendo B tramite uno XOR con il segnale di controllo e impostando il riporto iniziale C_0 a 1.

1.3 Estensione del Segno (Sign Extension)

Una caratteristica fondamentale dell'architettura RCAluG è l'estensione implicita del segno (Sign Extension). Durante il calcolo dei vettori P e G, gli operandi a n bit (A e B) vengono estesi al bit $n - esimo$ (MSB duplicato).

Questa tecnica garantisce che il risultato R, dimensionato a $n + 1$ bit, contenga sempre il valore matematico corretto dell'operazione, preservando l'informazione di segno anche qualora la somma di due numeri a n bit richieda una capacità di rappresentazione maggiore (evitando ambiguità di overflow nel risultato esteso).

Per soddisfare i requisiti di validazione e analisi delle risorse hardware, sono stati realizzati due moduli top-level "wrapper" che istanziano il core parametrico RCAluG, fissando il valore del generico n.

1.4 Modulo a 8 Bit (RCAlu)

L'entità RCAlu istanzia il core con il parametro $n = 8$.

- **Interfacce:** Ingressi a 8 bit, uscita a 9 bit ($n + 1$).
- **Scopo:** La dimensione ridotta permette l'esecuzione di una **simulazione esaustiva**. Il TestBench associato è in grado di generare e verificare tutte le combinazioni possibili di ingresso, garantendo la totale assenza di errori logici nel design.

1.5 Modulo a 16 Bit (RCAlu16)

L'entità RCAlu16 istanzia il core con il parametro $n = 16$.

- **Interfacce:** Ingressi a 16 bit, uscita a 17 bit $(n + 1)$.
- **Scopo:** Questo modulo rappresenta un dimensionamento tipico per data-path di elaborazione numerica. È destinato alla validazione tramite casi di test significativi (*corner cases*) e, soprattutto, all'analisi delle **performance hardware** (occupazione area FPGA e dissipazione di potenza), fornendo dati realistici su circuiti di dimensioni maggiori.

Per validare la correttezza funzionale dell'unità aritmetica e garantire il rispetto delle specifiche di progetto, sono stati sviluppati due ambienti di simulazione distinti (TestBench).

1.6 Verifica Esaustiva a 8 Bit (SimRCAlu)

Il primo TestBench, denominato SimRCAlu, è stato progettato per la verifica del modulo a 8 bit. Data la dimensione contenuta dello spazio degli stati, è stato possibile implementare un approccio algoritmico basato su cicli iterativi.

- **Metodologia:** Il processo di simulazione utilizza due cicli for annidati per generare gli operandi $A\ e\ B$, coprendo il range numerico da 0 a 127.
- **Copertura:** La simulazione viene eseguita in due fasi distinte:
  1. **Fase di Addizione ($\boldsymbol{Ctrl = 0}$):** Vengono testate le coppie di operandi in somma.
  2. **Fase di Sottrazione ($\boldsymbol{Ctrl = 1}$):** Vengono testate le medesime coppie in sottrazione.
- **Librerie:** È stata utilizzata la libreria IEEE.STD_LOGIC_ARITH per sfruttare la funzione CONV_STD_LOGIC_VECTOR, che facilita la conversione degli indici dei cicli interi in vettori logici da applicare alle porte del componente.

1.7 Verifica Funzionale a 16 Bit (SimRCAlu16)

Il secondo TestBench, SimRCAlu16, è dedicato alla validazione del modulo a 16 bit. A causa dell'elevato numero di combinazioni possibili (2^{32}), non è praticabile una simulazione esaustiva. L'approccio adottato è quello della verifica mirata tramite "Corner Cases".

- **Metodologia:** Gli stimoli sono assegnati manualmente per creare scenari specifici di interesse aritmetico.
- **Casi di Test Significativi:**
  - Operazioni standard di somma e differenza per verificare il corretto funzionamento di base.
  - **Verifica dei limiti (Overflow):** È stato incluso il caso $32767 + 1$ (massimo intero positivo a 16 bit $+$ 1). Questo test è cruciale per verificare che l'architettura gestisca correttamente il passaggio da un numero positivo massimo a un numero che richiede il diciassettesimo bit (o che risulterebbe negativo se troncato), validando la logica di estensione del segno implementata nel core RCAluG.

2. Implementazione del sommatore generico in VIVADO

L'implementazione del core aritmetico è stata realizzata attraverso un unico modulo VHDL parametrico, denominato RCAluG. Questa scelta progettuale sfrutta le capacità di astrazione del linguaggio per creare un componente flessibile, riutilizzabile per qualsiasi lunghezza n degli operandi.

2.1 Entity

```
1   library IEEE;
2   use IEEE.STD_LOGIC_1164.ALL;
3
4   entity RCAluG is
5       generic(n : integer := 8);
6       port(
7           A,B : in std_logic_vector(n-1 downto 0);
8           Ctrl : in std_logic;
9           R : out std_logic_vector(n downto 0)
10      );
11  end RCAluG;
```

La prima sezione di codice è suddivisa in due parti: L'importazione delle librerie utilizzate e la dichiarazione dell'entità.

Nelle prime due righe viene dichiarata la libreria IEEE e incluso il pacchetto standard STD_LOGIC_1164. Questo è fondamentale per utilizzare i tipi di dati std_logic e std_logic_vector, necessari per la descrizione dell'hardware digitale e la gestione dei segnali logici a più valori.

Viene poi definita l'entità del modulo con il nome RCAluG (che suggerisce "Ripple Carry ALU Generic"), dove è dichiarato un parametro generico n di tipo intero, inizializzato di default a 8. Questo rende il design modulare e riutilizzabile per vettori di diversa larghezza. Nella definizione delle porte di input e output possiamo usare il parametro n per la dichiarazione dei vettori: A e B (input) sono i due operandi ed R (output) è il vettore risultato. Notiamo che la sua dimensione è n downto 0 (quindi n+1 bit), il che permette di contenere il risultato esteso (inclusivo di eventuale riporto o bit di segno esteso) senza errori di overflow.

2.2 Architecture

```
13  architecture MyRCAluG of RCAluG is
14      signal B_Signed: std_logic_vector(n-1 downto 0);
15      signal Ctrl_vec: std_logic_vector(n-1 downto 0);
16      signal C: std_logic_vector(n+1 downto 0);
17      signal p, g: std_logic_vector(n downto 0);
18
19      begin
20          Ctrl_vec <= (others => Ctrl);
21          C(0) <= Ctrl;
22          B_Signed <= B xor Ctrl_vec;
23          p <= (A(n-1) xor B_Signed(n-1)) & (A xor B_Signed);
24          g <= (A(n-1) and B_Signed(n-1)) & (A and B_Signed);
25          C(n+1 downto 1) <= g or (p and C(n downto 0));
26          R <= p xor C(n downto 0);
27  end MyRCAluG;
```

Nell'architecture vengono creati i seguenti segnali interni:

- **B_Signed**: Vettore di supporto per gestire l'operando B. Servirà a contenere il valore di B oppure il suo complemento (not B), a seconda dell'operazione richiesta.
- **Ctrl_vec**: Vettore che replica il bit di controllo Ctrl su tutti i bit, utilizzato per le operazioni bit a bit (XOR).
- **C**: La catena dei riporti (Carry Chain). Ha dimensione n+1 per includere il carry-in iniziale e tutti i carry generati fino all'uscita.
- **P e G**: Vettori per i segnali di Propagate e Generate, estesi a n+1 bit per gestire correttamente l'aritmetica con estensione di segno.

Il segnale di ingresso Ctrl viene poi utilizzato per due scopi simultanei: Viene assegnato direttamente al bit meno significativo della catena dei riporti ($C(0)$) e assegna al segnale $B_Signed\ B$ o il suo negato tramite l'operazione logica $B \oplus Ctrl_vec$. Questo setup è fondamentale per l'implementazione del complemento a due, infatti se $Ctrl = 1$, l'operazione sarà una sottrazione in quanto bit di B vengono invertiti (complemento a uno) e, grazie al riporto iniziale $C(0)$ posto a 1, si completa la trasformazione in complemento a due (che matematicamente equivale a calcolare $-B$).

Procediamo calcolando vettori di propagazione (p) e generazione (g). Essi non vengono calcolati solo sugli n bit originali, ma vengono estesi a $n + 1\ bit$. Ciò avviene concatenando il bit più significativo (MSB) degli operandi ($A(n-1)$ e $B_Signed(n-1)$) con i vettori stessi. Questa operazione garantisce che l'aritmetica venga eseguita preservando correttamente il segno, permettendo al risultato finale R di avere una dimensione di $n + 1\ bit$ e di contenere eventuali overflow senza perdita di informazione.

Successivamente ricaviamo la catena dei riporti, descritta dall'equazione vettoriale per il segnale C. In un CLA questa formula viene espansa in modo che ogni riporto dipenda solo dagli input iniziali (A, B) e dal riporto iniziale C_0. Qui invece c'è una dipendenza diretta dai dati precedenti, quindi la logica implementa la classica formula del Ripple Carry: un riporto viene generato se i due bit sommati sono entrambi '1' (Generate), oppure se uno dei due è '1' e c'è un riporto dallo stadio precedente (Propagate). Infine, il risultato R viene ottenuto mediante l'operazione $p \oplus C$. Questo passaggio finale combina i bit di propagazione con i riporti calcolati, fornendo in uscita la somma (o differenza) corretta su $n + 1\ bit$.

3. Istanziazione dei Moduli Specifici (Wrapper) a 8 e 16 bit

L'architettura descritta sopra (RCAluG) è stata definita come un componente parametrico. Per soddisfare i requisiti di progetto, che richiedono analisi differenziate su due dimensionamenti distinti, sono state realizzate due entità dedicate, denominate Wrapper. Questi moduli non aggiungono logica aritmetica ulteriore, ma svolgono la funzione cruciale di istanziare il core parametrico configurandolo staticamente, così da poter differenziare gli ambienti di simulazione per il modulo a 8 bit e per quello a 16 bit e analizzare separatamente il livello hardware.

```
entity RCAlu8 is
    Port (
        A8, B8 : in STD_LOGIC_VECTOR (7 downto 0);
        R9: out STD_LOGIC_VECTOR (8 downto 0);
        Ctrl: in STD_LOGIC
     );
end RCAlu8;

architecture MyRCAlu8 of RCAlu8 is
    component RCAluG
        generic(n : integer);
        port(
            A,B : in std_logic_vector(n-1 downto 0);
            Ctrl : in std_logic;
            R : out std_logic_vector(n downto 0)
        );
    end component;

    begin
        RCAlu8: RCAluG generic map (8)
                port map(A8, B8, Ctrl, R9);
end MyRCAlu8;
```

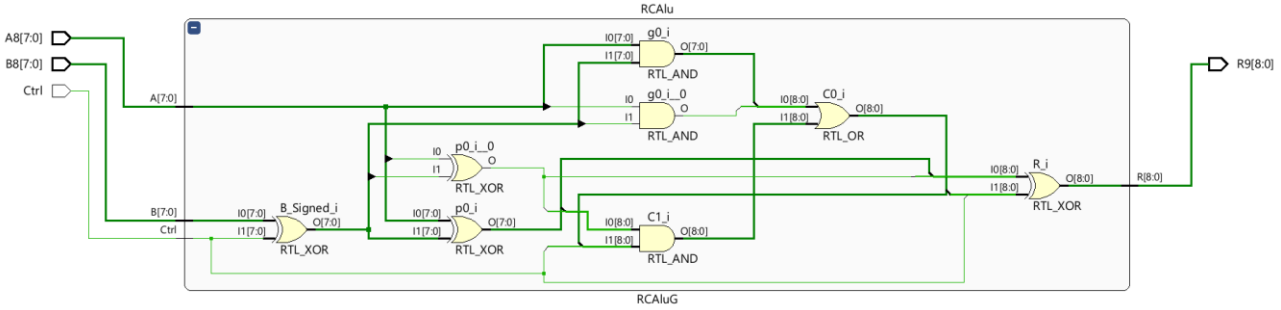
```
entity RCAlu16 is
    Port (
        A16, B16 : in std_logic_vector (15 downto 0);
        R17: out STD_LOGIC_VECTOR (16 downto 0);
        Ctrl: in STD_LOGIC
     );
end RCAlu16;

architecture MyRCAlu16 of RCAlu16 is
    component RCAluG
        generic(n : integer);
        port(
            A,B : in std_logic_vector(n-1 downto 0);
            Ctrl : in std_logic;
            R : out std_logic_vector(n downto 0)
        );
    end component;

    begin
        RCAlu16: RCAluG generic map (16)
                port map(A16, B16, Ctrl, R17);
end MyRCAlu16;
```

Viene dichiarato ed implementato il modulo RCAluG. Tramite la generic map vengono assegnati i valori 8 e 16 al parametro n. Questo istruisce il sintetizzatore a generare una versione fisica del circuito RCAluG dove ogni riferimento a n viene sostituito dal numero costante. Di conseguenza, i vettori interni e i loop del modulo generico si adattano automaticamente a questa dimensione. È necessaria poi la port map tradizionale che collega i segnali fisici di questa entità $(A8/16, B8/16, R9/17, Ctrl)$ alle porte del componente istanziato $(A, B, R, Ctrl)$.

A questo punto Vivado ci fornisce la possibilità di generare una rappresentazione grafica del circuito descritto:



4. Simulazione Behavioural

Per validare il funzionamento dell'unità aritmetica e soddisfare le richieste sono state sviluppate due simulazioni con l'obiettivo di stimolare il CUT, Circuit Under Test, attraverso una generazione esaustiva degli ingressi per quello ad 8 bit e con degli ingressi mirati per la versione a 16 bit.

4.1 Verifica esaustiva a 8 bit

Il primo test bench, 'SimRCAlu8', è stato progettato per la verifica del modulo a 8 bit. Data la dimensione contenuta dello spazio degli stati è stato possibile implementare un approccio algoritmico basato su cicli iterativi per coprire un'ampia gamma di combinazioni.

Vengono usati due cicli for annidati con indici ia e ib che variano da 0 a 127 coprendo l'intero range dei numeri positivi e negativi rappresentabili in complemento a due con 8 bit. La simulazione viene eseguita in due fasi distinte:

- **Fase di Addizione ($Ctrl = 0$):** Vengono testate le coppie di operandi in somma.
- **Fase di Sottrazione ($Ctrl = 1$):** Vengono testate le medesime coppie in sottrazione.

Viene usata la libreria 'IEE.STD_LOGIC_ARITH' e la funzione 'CONV_STD_LOGIC_VECTOR' per convertire dinamicamente gli indici interi dei cicli in vettori logici da applicare al componente.

```
entity SimRCAlu8 is
end SimRCAlu8;

architecture MySimRCAlu8 of SimRCAlu8 is
    component RCAlu8 is
        Port (
            A8, B8 : in STD_LOGIC_VECTOR (7 downto 0);
            R9: out STD_LOGIC_VECTOR (8 downto 0);
            Ctrl: in STD_LOGIC
         );
    end component;

    signal A8, B8: std_logic_vector(7 downto 0);
    signal R9: std_logic_vector(8 downto 0);
    signal Ctrl: std_logic;

begin
    CUT: RCAlu8 port map(A8, B8, R9, Ctrl);
    process begin
        Ctrl<='0';
        for ia in 0 to 127 loop
            A8<=CONV_STD_LOGIC_VECTOR(ia, 8);
            for ib in 0 to 127 loop
                B8<=CONV_STD_LOGIC_VECTOR(ib, 8);
                wait for 1 ns;
            end loop;
        end loop;

        Ctrl<='1';
        for ia in 0 to 127 loop
            A8<=CONV_STD_LOGIC_VECTOR(ia, 8);
            for ib in 0 to 127 loop
                B8<=CONV_STD_LOGIC_VECTOR(ib, 8);
                wait for 1 ns;
            end loop;
        end loop;
    end process;
end MySimRCAlu8;
```

4.2 Verifica funzionale a 16 bit

Il secondo test bench, 'SimRCAlu16', è dedicato alla validazione del modulo a 16 bit. A causa dell'elevato numero di combinazioni possibili, 2^{32}, una simulazione esaustiva risulterebbe computazionalmente troppo onerosa. L'approccio adottato è quello della verifica mirata tramite dei casi limite.

I valori di ingresso sono stati assegnati manualmente per poter creare degli specifici scenari e vengono distanziati l'uno dall'altro da $10\ ns$.

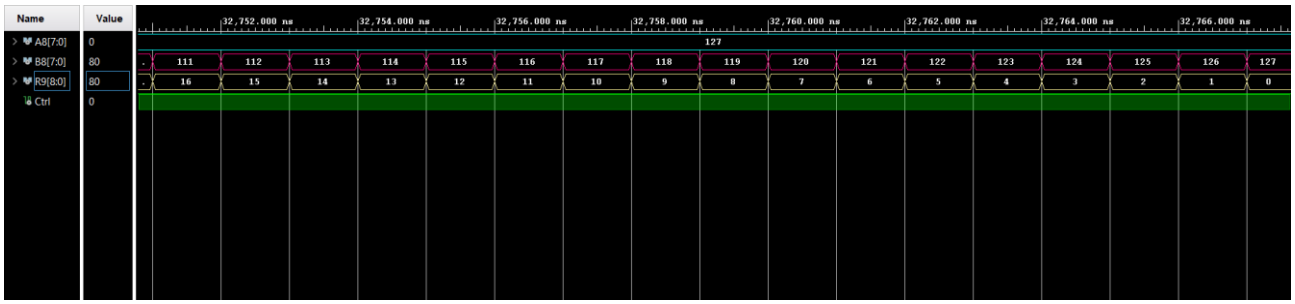
Sono stati simulati due scenari:

- **Caso standard**: $29 + 16$ o $15 + 18$ ad esempio, per verificare il funzionamento di base;
- **Caso overflow**: $32767 + 1$, in questo caso si somma 1 al massimo numero intero positivo rappresentabile con 16 bit. È un test molto importante in quanto si può dimostrare che il risultato non subisce overflow ma viene rappresentato correttamente.

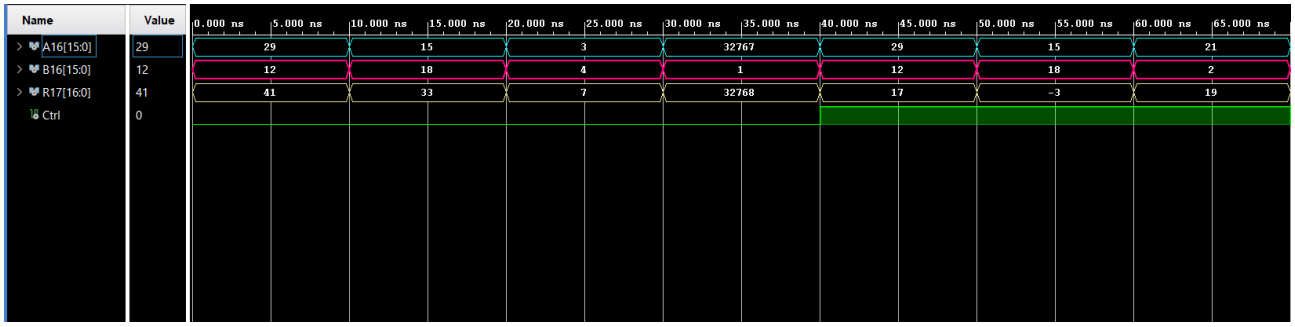
```
5   entity SimRCAlu16 is
6   end SimRCAlu16;
7
8   architecture MySimRCAlu16 of SimRCAlu16 is
9       component RCAlu16 is
10          Port (
11              A16, B16 : in std_logic_vector (15 downto 0);
12              R17: out STD_LOGIC_VECTOR (16 downto 0);
13              Ctrl: in STD_LOGIC
14           );
15      end component;
16      signal A16, B16: std_logic_vector(15 downto 0);
17      signal R17: std_logic_vector(16 downto 0);
18      signal Ctrl: std_logic;
19
20  begin
21      CUT: RCAlu16 port map(A16, B16, R17, Ctrl);
22      process begin
23          Ctrl<='0';
24          A16 <= CONV_STD_LOGIC_VECTOR(29, 16);
25          B16 <= CONV_STD_LOGIC_VECTOR(12, 16);
26          wait for 10 ns;
27          A16 <= CONV_STD_LOGIC_VECTOR(15, 16);
28          B16 <= CONV_STD_LOGIC_VECTOR(18, 16);
29          wait for 10 ns;
30          A16 <= CONV_STD_LOGIC_VECTOR(3, 16);
31          B16 <= CONV_STD_LOGIC_VECTOR(4, 16);
32          wait for 10 ns;
33          A16 <= CONV_STD_LOGIC_VECTOR(32767, 16);
34          B16 <= CONV_STD_LOGIC_VECTOR(1, 16);
35          wait for 10 ns;
36
37          Ctrl<='1';
38          A16 <= CONV_STD_LOGIC_VECTOR(29, 16);
39          B16 <= CONV_STD_LOGIC_VECTOR(12, 16);
40          wait for 10 ns;
41          A16 <= CONV_STD_LOGIC_VECTOR(15, 16);
42          B16 <= CONV_STD_LOGIC_VECTOR(18, 16);
43          wait for 10 ns;
44          A16 <= CONV_STD_LOGIC_VECTOR(21, 16);
45          B16 <= CONV_STD_LOGIC_VECTOR(2, 16);
46          wait for 10 ns;
47      end process;
48  end MySimRCAlu16;
```

4.3 Risultati delle simulazioni

Risultati del test ad 8 bit dopo 32,768 μs di esecuzione



Risultati del test a 16 bit



5. Reports

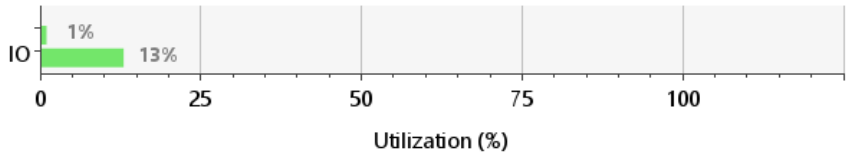
In questa sezione si analizza come è stato mappato il design sulle risorse del dispositivo.

5.1 Utilization Report

L'analisi dell'utilizzo delle risorse è fondamentale per comprendere quando un design sia efficiente e scalabile.

- **Risorse combinatorie**: Dato che l'unità aritmetica è stata progettata come circuito combinatorio è presente un utilizzo minimo, quasi nullo, delle risorse sequenziali come Flip-Flop o registri. L'implementazione si concentra sull'uso di LUTs, Look Up Tables, e delle catene di carry dedicate per garantire la massima velocità di propagazione del segnale.
- **Blocchi di input/output**: Il report mostra un utilizzo del 13% per i blocchi di Input/Output Buffer. 26 utilizzati su 200 disponibili. Perfettamente coerente per $n = 8\ bit$, in quanto abbiamo $A = 8\ bit +\ B = 8\ bit +\ ctrl = 1\ bit + R = 9\ bit$.

Resource	Utilization	Available	Utilization %
LUT	11	53200	0.02
IO	26	200	13.00

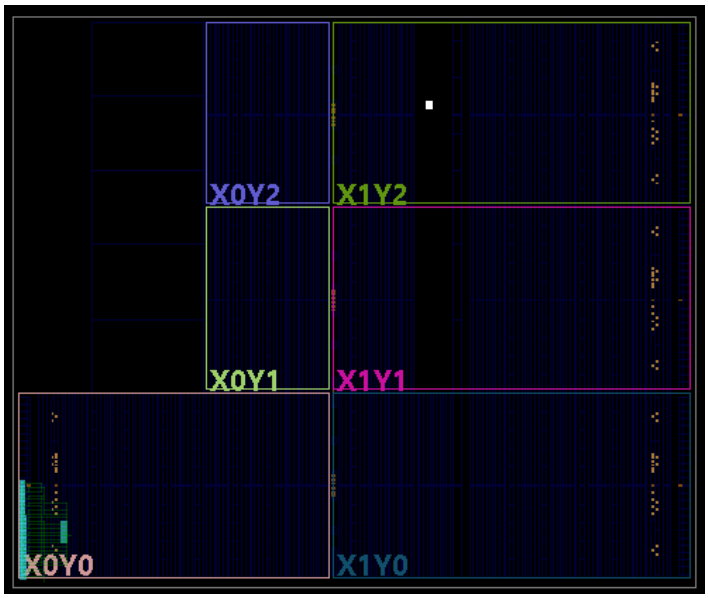


L'occupazione è molto bassa rispetto alla capacità totale confermando che un sommatore del tipo Ripple Carry Adder è molto efficiente in termini di area.

5.2 Device View

L'immagine del device mostra come il circuito è stato posizionato sul chip.

- **Compattezza e localizzazione**: l'ALU è stata mappata in modo compatto concentrando le risorse logiche in un'area limitata del chip, principalmente la regione $X0Y0$.
- **Vantaggi dell'implementazione**: questa localizzazione compatta fa pensare ad un routing efficiente delle interconnessioni. Un routing breve e ben localizzato minimizza ritardi di propagazione e riduce il consumo di potenza dinamica legato alla guida di segnali su lunghe distanze.



5.3 Power Report

La potenza totale consumata dal circuito (P_{tot}) è la somma della potenza statica ($P_{statica}$) e della potenza dinamica ($P_{dinamica}$). Sul chip è presente una potenza stimata pari a $6.491\ W$, nonostante il consumo apparente il report segnala una temperatura di giunzione di $99.9°\ C$ e un margine termico negativo di $-14.9°\ C$. Questo indica che il design, nelle condizioni simulate, supera i limiti di temperatura ammessi dal dispositivo e quindi non è termicamente sostenibile con le condizioni attuali.

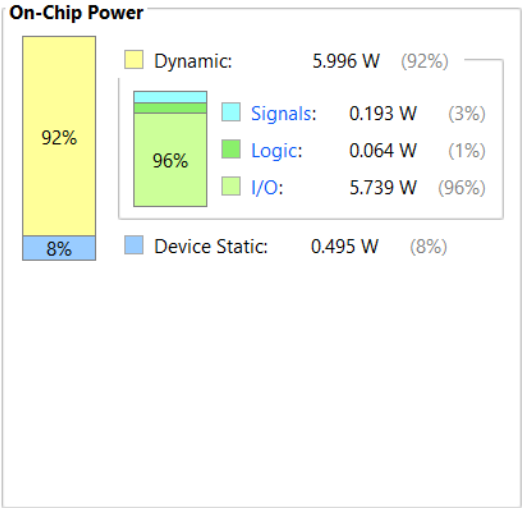
L'analisi della ripartizione della potenza spiega la causa del consumo elevati, in quanto:

- **Potenza statica**: Ammonta a $0.495\ W$ del totale, quindi l'8%. Questa è la potenza di dispersione di base del chip e dipende principalmente dalla tecnologia del dispositivo.
- **Potenza dinamica**: Rappresenta la maggior parte del consumo, $5.379\ W$ quindi il 92%. È generata dall'attività di commutazione dei segnali.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**6.491 W (Junction temp exceeded!)**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**99,9°C**
Thermal Margin:	-14,9°C (-1,1 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	11,5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity



La ripartizione della potenza dinamica è particolarmente rivelatrice in quanto la quasi totalità della potenza dinamica è assorbita dai blocchi di I/O, ciò indica che l'eccessivo consumo non deriva dall'ALU ma dal circuito che pilota i pin di uscita. Inoltre, il consumo diretto dell'ALU è trascurabile e quindi conferma che il circuito combinatorio è molto efficiente dal punto di vista energetico.

L'elevato consumo di I/O e la conseguente temperatura di giunzione critica (99.9°C) dovuti alla sovrastima fatta da Vivado in assenza di vincoli fisici sul carico dei pin (constraints). Il tool sovrastima drasticamente la corrente necessaria per pilotare le uscite.

Generalmente possiamo migliorare il consumo eccessivo intervenendo sulla frequenza di aggiornamento del circuito.

5.4 Conclusioni sui reports

La valutazione, quindi rivela che l'implementazione è logicamente molto efficiente, ma la configurazione operativa induce un rischio termico importante.